TECNOLÓGICO
NACIONAL DE MÉXICO

TAREA 4.2: PROCESO PASO A PASO DE NORMALIZACIÓN DEL PROYECTO APLICADO

Asignatura: Sistemas de Bases de Datos

Docente: Ruben Miguel Riojas Rodríguez

Alumno: Eduardo Salazar Martínez

1. Contextualización del Problema y Estado Desnormalizado (Estado 0)

En los sistemas de información reales, la recolección inicial de datos suele realizarse mediante estructuras monolíticas desnormalizadas u hojas de cálculo. En nuestro proyecto aplicado, la tabla **dbo.REGISTRO_COMPRA_UNIFICADO** presenta severas violaciones de diseño relacional, almacenando datos de clientes, productos, pedidos y proveedores en un solo registro con listas multivaluadas.

```
-- ============================================================
-- SCRIPT ESTADO 0: ESTRUCTURA DESNORMALIZADA (script_no_normalizado-v2.sql)
-- Entorno: Microsoft SQL Server (T-SQL)
-- ============================================================
IF OBJECT_ID('dbo.REGISTRO_COMPRA_UNIFICADO', 'U') IS NOT NULL
    DROP TABLE dbo.REGISTRO_COMPRA_UNIFICADO;
GO

CREATE TABLE dbo.REGISTRO_COMPRA_UNIFICADO (
    ID_Pedido INT,
    Fecha_Pedido VARCHAR(20),
    ID_Cliente INT,
    Nombre_Cliente VARCHAR(100),
    Email_Cliente VARCHAR(100),
    Direccion_Cliente VARCHAR(150),
    ID_Productos VARCHAR(255),      -- Lista multivaluada: "101, 102"
    Nombres_Productos VARCHAR(255), -- Lista multivaluada: "Laptop, Mouse"
    Precios_Unitarios VARCHAR(255), -- Lista multivaluada: "1200.00, 25.00"
    Cantidades VARCHAR(255),        -- Lista multivaluada: "1, 2"
    ID_Proveedor INT,
    Nombre_Proveedor VARCHAR(100),
    Email_Proveedor VARCHAR(100),
    Monto_Total_Pedido DECIMAL(10,2)
);
GO

INSERT INTO dbo.REGISTRO_COMPRA_UNIFICADO VALUES
(1001, '2026-10-15', 501, 'Carlos Mendoza', 'carlos@email.com', 'Av. Reforma 123',
 '101, 102', 'Laptop Dell, Mouse Logitech', '1200.00, 25.00', '1, 2',
 901, 'Tech Supplies SA', 'contacto@techsupplies.com', 1250.00);
GO
```

2. Proceso de Normalización Paso a Paso

Fase 1: Transición de Estado 0 a Primera Forma Normal (1FN)

Regla aplicada: Eliminación de grupos repetitivos y garantía de atomicidad de atributos. Las listas compuestas como ID_Productos ('101, 102') se descomponen de modo que cada celda contenga un único valor indivisible. Se establece la clave primaria compuesta formada por (ID_Pedido, ID_Producto).

Dependencia Funcional en 1FN:

```
{ID_Pedido, ID_Producto} -> {Fecha_Pedido, ID_Cliente, Nombre_Cliente, Email_Cliente,
Direccion_Cliente, Nombre_Producto, Precio_Unitario, Cantidad, ID_Proveedor, Nombre_Proveedor,
Email_Proveedor}
```

Fase 2: Transición de 1FN a Segunda Forma Normal (2FN)

Regla aplicada: Eliminación de Dependencias Funcionales Parciales. Ningún atributo no clave debe depender de un subconjunto propio de la clave primaria compuesta (ID_Pedido, ID_Producto).

Atributo / Dependencia	Tipo de Dependencia en 1FN	Acción de Descomposición (2FN)
Fecha_Pedido, ID_Cliente, Datos Cliente	Parcial de ID_Pedido	Se extrae a la tabla PEDIDO_2FN
Nombre_Producto, Precio, Datos Proveedor	Parcial de ID_Producto	Se extrae a la tabla PRODUCTO_2FN
Cantidad, Precio_Venta	Completa de {ID_Pedido, ID_Producto}	Permanecen en DETALLE_PEDIDO_2FN

Fase 3: Transición de 2FN a Tercera Forma Normal (3FN) y Validación FNBC

Regla aplicada: Eliminación de Dependencias Transitivas. Ningún atributo no clave puede depender de otro atributo no clave.

• En PEDIDO_2FN: ID_Pedido -> ID_Cliente -> {Nombre_Cliente, Email_Cliente, Direccion_Cliente} (Transitiva). Se extrae la entidad CLIENTE.

• En PRODUCTO_2FN: ID_Producto -> ID_Proveedor -> {Nombre_Proveedor, Email_Proveedor} (Transitiva). Se extrae la entidad PROVEEDOR.

Validación FNBC (Boyce-Codd): En el esquema final de 5 tablas (PROVEEDOR, CLIENTE, PRODUCTO, PEDIDO, DETALLE_PEDIDO), todo determinante X -> Y es una Superclave / Clave Candidata explícita. Por ende, el esquema satisface rigurosamente FNBC.

3. Matriz Analítica de Anomalías Corregidas

Tipo de Anomalía	Comportamiento en Estado 0 (Desnormalizado)	Solución en Esquema Totalmente Normalizado (3FN/FNBC)
Inserción	Imposible dar de alta un nuevo cliente o producto sin crear un pedido ficticio.	Permite registrar independientes en dbo.CLIENTE o dbo.PRODUCTO sin requerir pedidos.
Borrado	Eliminar un pedido provoca la pérdida permanente de datos de clientes y proveedores.	Eliminar un registro en dbo.PEDIDO borra solo sus ítems asociados en dbo.DETALLE_PEDIDO.
Modificación	Cambiar el email de un proveedor requiere actualizar múltiples filas, generando datos inconsistentes.	La actualización se realiza en un único punto en dbo.PROVEEDOR, propagándose vía ON UPDATE CASCADE.

4. Script SQL DDL Corregido (SQL Server / SQL Express - T-SQL)

A continuación se presenta el código T-SQL de la base de datos totalmente normalizada, coincidente en su totalidad con el archivo ejecutable normalizacion_proyecto-v2.sql.

```
-- ============================================================
-- SCRIPT ESQUEMA NORMALIZADO EN 3FN / FNBC (normalizacion_proyecto-v2.sql)
-- Entorno: Microsoft SQL Server / SQL Express (T-SQL)
-- ============================================================
IF OBJECT_ID('dbo.DETALLE_PEDIDO', 'U') IS NOT NULL DROP TABLE dbo.DETALLE_PEDIDO;
IF OBJECT_ID('dbo.PEDIDO', 'U') IS NOT NULL DROP TABLE dbo.PEDIDO;
IF OBJECT_ID('dbo.PRODUCTO', 'U') IS NOT NULL DROP TABLE dbo.PRODUCTO;
IF OBJECT_ID('dbo.CLIENTE', 'U') IS NOT NULL DROP TABLE dbo.CLIENTE;
IF OBJECT_ID('dbo.PROVEEDOR', 'U') IS NOT NULL DROP TABLE dbo.PROVEEDOR;
GO

CREATE TABLE dbo.PROVEEDOR (
    ID_Proveedor INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_Proveedor VARCHAR(100) NOT NULL,
    Email_Proveedor VARCHAR(100) NOT NULL UNIQUE,
    Telefono VARCHAR(20)
);
GO

CREATE TABLE dbo.CLIENTE (
    ID_Cliente INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_Cliente VARCHAR(100) NOT NULL,
    Email_Cliente VARCHAR(100) NOT NULL UNIQUE,
    Direccion_Cliente VARCHAR(150) NOT NULL
);
GO

CREATE TABLE dbo.PRODUCTO (
    ID_Producto INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_Producto VARCHAR(100) NOT NULL,
    Precio_Unitario DECIMAL(10,2) NOT NULL CHECK (Precio_Unitario >= 0),
    ID_Proveedor INT NOT NULL,
    CONSTRAINT FK_Producto_Proveedor FOREIGN KEY (ID_Proveedor)
        REFERENCES dbo.PROVEEDOR(ID_Proveedor)
        ON DELETE NO ACTION ON UPDATE CASCADE
```

```
);
GO

CREATE TABLE dbo.PEDIDO (
    ID_Pedido INT IDENTITY(1,1) PRIMARY KEY,
    Fecha_Pedido DATE NOT NULL,
    ID_Cliente INT NOT NULL,
    CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ID_Cliente)
        REFERENCES dbo.CLIENTE(ID_Cliente)
        ON DELETE NO ACTION ON UPDATE CASCADE
);
GO

CREATE TABLE dbo.DETALLE_PEDIDO (
    ID_Pedido INT NOT NULL,
    ID_Producto INT NOT NULL,
    Cantidad INT NOT NULL CHECK (Cantidad > 0),
    Precio_Venta DECIMAL(10,2) NOT NULL CHECK (Precio_Venta >= 0),
    CONSTRAINT PK_DetallePedido PRIMARY KEY (ID_Pedido, ID_Producto),
    CONSTRAINT FK_Detalle_Pedido FOREIGN KEY (ID_Pedido)
        REFERENCES dbo.PEDIDO(ID_Pedido)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT FK_Detalle_Producto FOREIGN KEY (ID_Producto)
        REFERENCES dbo.PRODUCTO(ID_Producto)
        ON DELETE NO ACTION ON UPDATE NO ACTION
);
GO
```

5. Conclusiones y Validación de la Práctica

El proceso de normalización aplicado transformó una estructura monolítica ineficiente en un esquema relacional optimizado de 5 entidades en 3FN / FNBC para SQL Server Express. Esto garantiza el cumplimiento estricto de los estándares de integridad referencial, elimina la redundancia y asegura que la base de datos sea escalable, mantenible y libre de las tres anomalías relacionales clásicas.